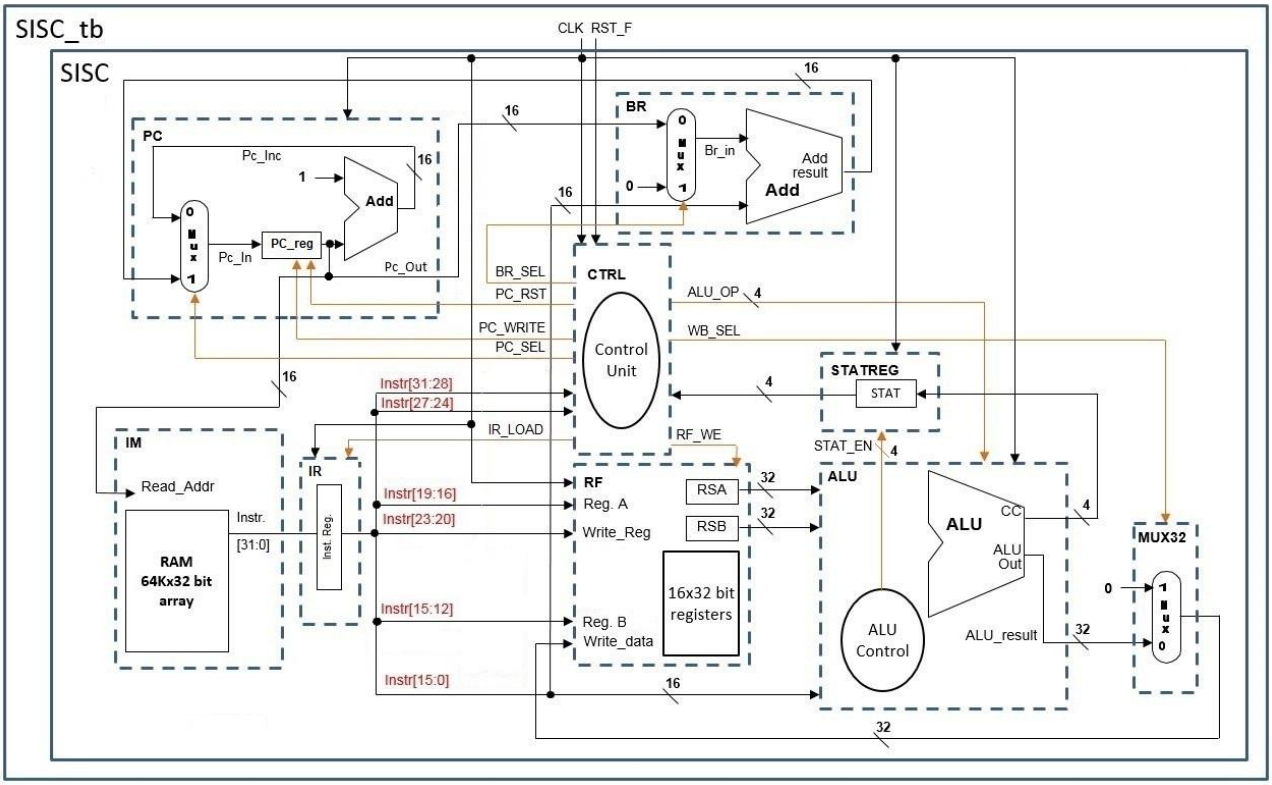
ECE:3350 Spring 2026 - Computer Architecture and Organization

Simple Instruction Set Computer (SISC) Project

Part 2: Due Monday, April 6, at 11:59 pm

For this part of the project, you are to include in the Datapath the modules for program counter control, instruction memory, and branch execution, as shown below.



You are given:

- Your own solution from Part 1. We will not provide a working solution to build off of, so **it is imperative that you finish Part 1 before moving on to Part 2.**
- An imem.data file that contains all of the instructions from Part 1. You are provided this so that you may be certain that your modifications from Part 1 to Part 2 have not broken the original functionality. (See the note about imem.data below for more information.)
- The Verilog files pc.v, br.v, ir.v, and im.v (with descriptions provided), which you are not to modify.

You are required to:

- Modify the control unit to generate the new BR_SEL, PC_RST, PC_WRITE, PC_SEL, and IR_LOAD signals, while not disrupting the control signals from Part 1.
- Modify the sisc module to instantiate and connect the pc, br, ir, and im modules.
- Compress your project folder and submit it to the "Verilog Project – Part 2" ICON dropbox.

Part 2 Notes:

- You are given a new testbench file that only generates the clock and reset signals.
- **About imem.data:** When the simulation of your design begins, the im module reads the data in the file imem.data and stores it. The data should be written in 32-bit hexadecimal strings (8 hexadecimal digits each), sequentially from address 00 onward. In-line comments can be included if they are preceded by two forward slash characters (//Just like in C). You may want to keep the original contents of imem.data as a reference for how to format the instructions. The imem.data file also contains new instructions to test that the branch instructions work correctly.
- You will no longer have to control the timing of the instructions with delays. If your control unit correctly generates the program counter and branch signals, the instructions will be read during the instruction fetch stage by the im module.
- **Be careful when implementing the PC update and branch control signals!** The PC value is to be incremented as part of the Fetch state. This incremented value of the PC is then output to the BR module for use during the decode state to determine the next PC value if a branch is taken. Please read the section in the instruction set that describes the branch instructions operation. All of the implementation of the branch instructions is best done in the Decode state.
- Again, you may use any $monitor(…) statements you wish to test your design, but your submitted sisc.v should monitor the following signals: IR, PC, R1, R2, R3, ALU_OP, BR_SEL, PC_WRITE, and PC_SEL.

Submission Overview:

- Your .zip file should be named "yourprojectname_p2.zip" and contain the following:
    - Alu.v, br.v, im.v, mux32.v, pc.v, ir.v, rf.v, and statreg.v, exactly as they were provided.
    - Ctrl.v, and sisc.v completed by your group.
        - Again, sisc.v should contain the $monitor statement described above.
    - The 'work' directory.
    - The transcript information from your simulation.

Grading Rubric:

Correct Execution of Pt. 1 Instructions	20 pts
Correct Execution of Pt. 2 Instructions	30 pts
Ctrl.v Implementation	30 pts
Imem.data Testing all Pt. 2 Instructions	20 pts
Total	**100 pts**